Middleware: the pipeline, next(), and order

9 min read · Updated 24 Jul 2026 · Topic 36 of 70

Bookmark · Mark done

Why it matters

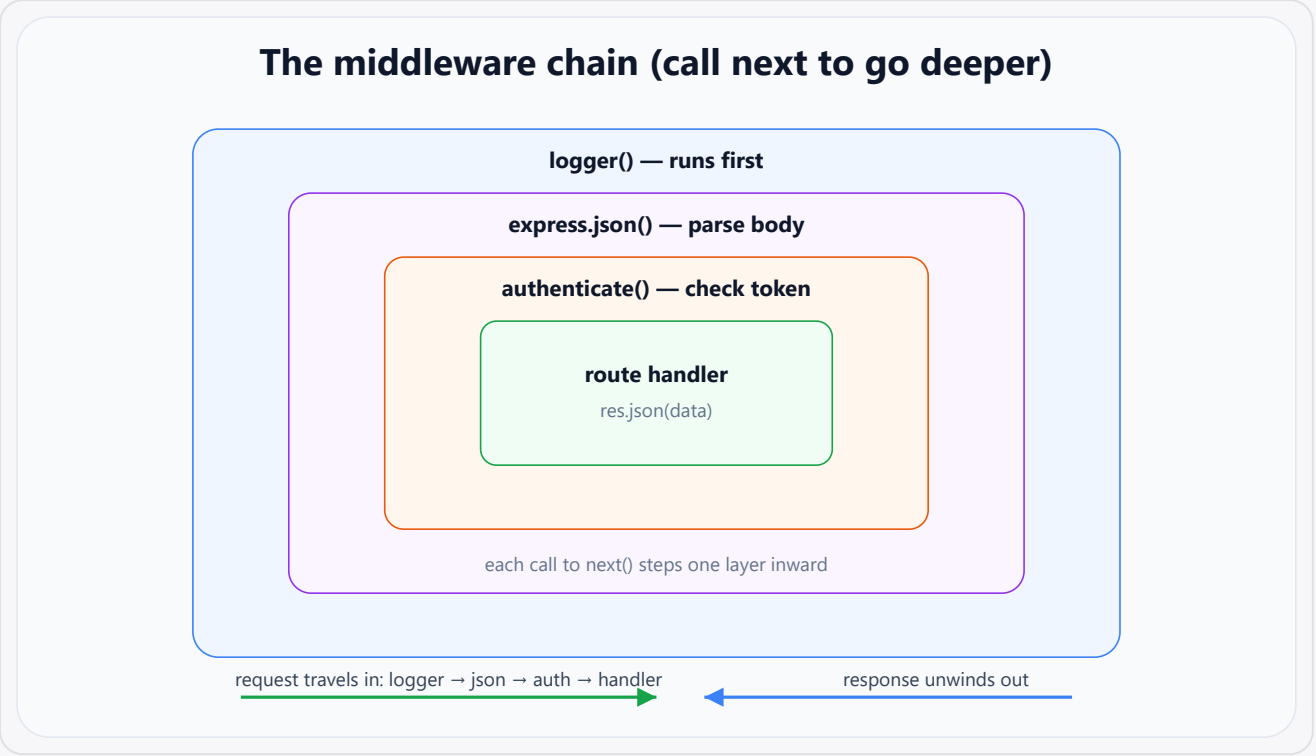
Middleware is the single most important concept in Express — more than routing, more than the helpers. Everything Express does is expressed as middleware: parsing bodies, logging, authenticating, setting CORS headers, handling errors, and running your route logic are all just functions in one ordered pipeline. If routing answers "which code runs?", middleware answers "what happens before, during, and after it runs?". Interviewers probe middleware because a candidate who truly understands it understands the whole framework; someone who has only copy-pasted `app.use(cors())` usually stumbles the moment you ask *why* order matters or *what* `next()` *actually does*.

Practically, middleware is how you avoid repeating yourself. Instead of adding an auth check, a request log, and a body parse to every one of forty handlers, you write each once and register it in the pipeline. This topic explains the pipeline model precisely — the "onion", the role of `next()`, why registration order is behaviour, and the difference between application-level and router-level middleware — so that the error-handling topic that follows (which is also just middleware) feels like a natural extension rather than a new rule.

The concept, simply

A middleware is **a function with the signature** `(req, res, next)`. Express calls it with the request object, the response object, and a `next` function. Inside, the middleware can do three kinds of thing: *read or modify* `req` / `res` (attach a user, set a header, log a line), *end the request* by sending a response, or *pass control onward* by calling `next()`. A middleware that neither responds nor calls `next()` hangs the request forever — the

You register middleware with `app.use(fn)` to run it on every request, or `app.use('/path', fn)` to scope it to paths starting with `/path`, or as extra arguments on a route (`app.get('/x', authCheck, handler)`) to run it only for that route. Express keeps all of these in one ordered stack, in the order you register them. When a request comes in, it enters the top of the stack and flows down, function by function, each one deciding whether to pass control along.

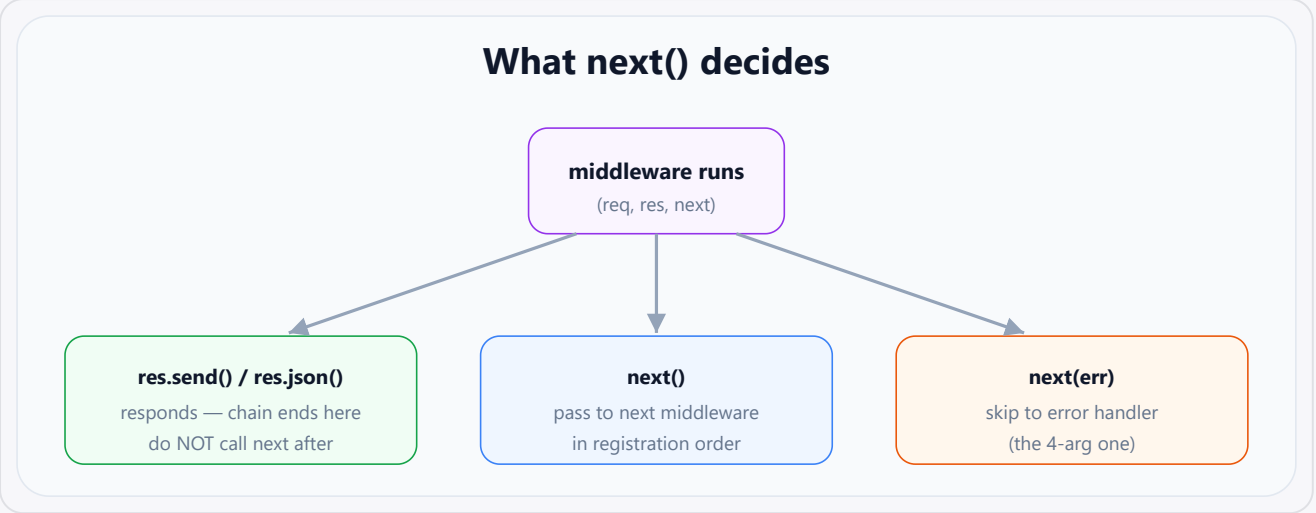


Middleware nests like an onion: each next() steps one layer inward toward the handler, and the response unwinds back out.

The classic image is an **onion**. The request enters through the outer layers (logger, then body parser, then auth), each calling `next()` to reach the next layer in, until it hits the route handler at the core. The response then travels back out through any code the middleware placed *after* its `next()` call. This "in through the layers, out through the layers" shape is why some middleware can measure request duration: it records a timestamp before `next()` and computes the elapsed time after control returns.

How it works

Express stores middleware in an ordered array. Processing a request means walking that array with an internal cursor. Express calls the first matching entry; that entry receives a `next` function which, when invoked, advances the cursor and calls the next matching entry — and so on. **Calling `next()` is literally "run the next middleware".** If a middleware never calls `next()` and never responds, the cursor never advances and the request stalls. If it calls `next()` more than once, later middleware may run twice and you can hit "headers already sent" errors when two of them try to respond.



next() is a control decision: respond and stop, call next() to continue, or call next(err) to jump to the error handler.

The `next` function has a special overload: if you call it **with an argument** — `next(err)` — Express treats that as an error and skips every remaining *normal* middleware, jumping straight to the *error-handling* middleware (the ones with four parameters, covered next topic). There is one reserved string, `next('route')`, which skips the rest of the handlers for the *current* route and moves to the next matching route — a niche tool. Everything else you pass to `next` is an error.

Because the stack is ordered and Express walks it linearly, **registration order is behaviour, not style**. Body parsers must be registered before routes that read `req.body`, or the body is unparsed when the handler runs. Auth middleware must come before the protected routes. A logger placed at the very top sees every request; placed at the bottom it sees only requests that fell through everything else. There is no dependency graph or priority system — just position in the array.

Middleware comes in scopes. **Application-level** middleware is registered on `app` (`app.use(fn)`) and applies to the whole app. **Router-level** middleware is registered on

passed inline to a single route. **Built-in** middleware ships with Express (`express.json()`, `express.urlencoded()`, `express.static()`). **Third-party** middleware is installed from npm (`cors`, `helmet`, `morgan`, `express-rate-limit`). All of them are the same shape of function in the same kind of stack.

Code examples

1. A custom logging middleware that times the request using the onion shape.

JAVASCRIPT Copy

```
app.use((req, res, next) => {
  const start = Date.now();
  console.log(`→ ${req.method} ${req.url}`);
  res.on('finish', () => {              // runs when the response is sent
    console.log(`← ${res.statusCode} (${Date.now() - start}ms)`);
  });
  next();                               // must call next, or the request h
});
```

2. Authentication as middleware, attaching data to `req` for later handlers.

JAVASCRIPT Copy

```
function requireAuth(req, res, next) {
  const token = req.get('Authorization');
  if (!token) {
    return res.status(401).json({ error: 'No token' }); // ends the chain
  }
  req.user = decode(token);             // pass data downstream via req
  next();                               // continue to the handler
}

app.get('/profile', requireAuth, (req, res) => {
  res.json({ user: req.user });         // req.user was set by the middlewa
});
```

3. Application-level vs router-level scope.

JAVASCRIPT Copy

```
// app-level: every request
app.use(express.json());

// router-level: only requests under /admin
const admin = express.Router();
admin.use(requireAuth);                // guards the whole admin subtree
admin.get('/stats', (req, res) => res.json({ ok: true }));
app.use('/admin', admin);
```

4. **Common third-party stack**, in the order they should run.

JAVASCRIPT Copy

```
const helmet = require('helmet');
const cors = require('cors');
const morgan = require('morgan');

app.use(helmet());                     // security headers first
app.use(cors());                       // CORS headers
app.use(morgan('dev'));                // request logging
app.use(express.json());               // body parsing
// ... routes registered AFTER the shared middleware ...
```

Common mistakes & gotchas

- **Forgetting** `next()`. A middleware that neither responds nor calls `next()` hangs the request silently — no error, the client just times out. The most common Express bug.
- **Calling** `next()` **AND responding.** If you `res.send()` and then call `next()`, downstream code may try to respond again → "Cannot set headers after they are sent". Return after responding: `return res.status(401).json( ... )`.
- **Wrong order.** Registering `express.json()` after a route means that route sees `req.body === undefined`. Shared middleware goes first.

- **Doing heavy sync work in middleware.** It runs on every matching request; a blocking operation there throttles the whole server.
- **Assuming `app.use('/api', fn)` matches only `/api`.** It matches every path that *starts with* `/api` (`/api/users`, `/api/x/y`), which is usually what you want but occasionally surprises.
- **Mutating `req` / `res` unpredictably.** Attaching data is fine (`req.user`), but overwriting core properties or relying on a property another middleware may not have set leads to brittle chains.

Best practices

- **Fix a registration order and stick to it:** security headers → CORS → logging → body parsing → app routes → 404 → error handler.
- **Always `return` after sending a response** inside middleware, so control cannot accidentally fall through to `next()`.
- **Scope middleware as narrowly as it needs to be** — route-level for one endpoint, router-level for a subtree, app-level only for truly global concerns.
- **Keep each middleware single-purpose** (log, or parse, or authenticate — not all three), so the pipeline is readable and reusable.
- **Pass data downstream via `req`** (e.g. `req.user`, `req.requestId`) rather than globals; it stays request-scoped and safe under concurrency.
- **Prefer battle-tested third-party middleware** (`helmet`, `cors`, `express-rate-limit`, `morgan`) over hand-rolled versions of the same concern.

⭐ Interview lens

> **Q: "What is middleware in Express?"**
> A: A function of the form `(req, res, next)` that runs during the request pipeline. It can read/modify the request and response, end the request by responding, or pass control to the next function with `next()`. Every part of Express — parsing, auth, logging, even route handlers and error handlers — is middleware.

A: `next()` advances Express to the next matching middleware in the stack. If a middleware neither responds nor calls `next()`, the request hangs indefinitely — Express never moves the cursor forward, so the client just waits and times out.

Q: "Why does middleware order matter?"

A: Express walks the middleware stack in registration order, so position *is* behaviour. Body parsers must precede routes that read `req.body`; auth must precede protected routes; a top-level logger sees every request. There is no priority system — only array order.

Q: "What is the difference between `next()` and `next(err)`?"

A: `next()` continues to the next normal middleware. `next(err)` — with any argument — signals an error: Express skips all remaining normal middleware and jumps to the error-handling middleware (the four-argument ones). `next('route')` is a special case that skips to the next matching route.

Q: "Application-level vs router-level middleware?"

A: App-level is registered on `app` and runs for the whole app; router-level is registered on an `express.Router()` and runs only for requests that reach that router. Router-level is how you guard one resource subtree — e.g. `adminRouter.use(requireAuth)` — without affecting unrelated routes.

Q (trap): "You call `res.json(data)` and then `next()` in the same middleware. What happens?"

A: You have already sent the response, so when a downstream handler also tries to write, Express throws "Cannot set headers after they are sent to the client". Always `return` after responding; do not fall through to `next()`.

- Middleware = `(req, res, next) => { ... }`. Read/modify, respond, or `next()`.
- No `next()` and no response = **request hangs**. Respond and then don't `next()`.
- `next()` = next middleware; `next(err)` = jump to error handler; `next('route')` = next matching route.
- **Order = behaviour.** Shared middleware before routes.
- Scopes: app (`app.use`), router (`router.use`), route (inline arg), built-in, third-party.
- Onion model: request flows in via `next()`, response unwinds out.
- Pass data downstream on `req` (e.g. `req.user`), never on globals.
- Recommended order: helmet → cors → logger → `express.json()` → routes → 404 → error handler.

QUICK CHECK — no marks, just for you.

1. What happens if a middleware neither sends a response nor calls next()?

- ◯ Express automatically calls next() after a short timeout
- ◯ The request hangs indefinitely — Express never advances its cursor, so the client just waits and eventually times out
- ◯ Express throws an error and skips to the error handler
- ◯ The next matching route runs anyway after the middleware returns

Check answer

2. What is the difference between next() and next(err)?

- ◯ next() and next(err) both continue to the next normal middleware
- ◯ next(err) retries the current middleware with the error attached

- next() continues to the next normal middleware; next(err) skips all remaining normal middleware and jumps to the four-argument error handler
- next(err) immediately ends the process with the error

Check answer

3. Why does the order in which you register middleware matter in Express?

- Order is only stylistic; Express sorts middleware by priority internally
- Order matters only for error handlers, not normal middleware
- Express runs middleware alphabetically by function name
- Express walks the stack in registration order, so position is behaviour: body parsers must precede routes that read req.body, auth must precede protected routes

Check answer

Previous
Routing: params, query, and Routers

Next
Error handling: the 4-arg handler and async errors

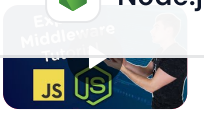
LEARN MORE

ARTICLES & DOCS

- Using middleware
  expressjs.com
- Writing middleware for use in Express apps
  expressjs.com

VIDEOS

Node.js Backend Kit



Learn Express Middleware In 14 Minutes

Web Dev Simplified

Hand-picked references for this lesson.